



Introdução e Avisos.....	3
Trabalhando com SQL.....	3
Criar Banco de Dados.....	3
Deletar Banco de Dados.....	3
Criar Tabela.....	4
ATENÇÃO!.....	5
Deletar Tabela.....	5
Inserir Dados na Tabela.....	5
Atualizar Estrutura da Tabela.....	6
Adicionar Coluna.....	6
Modificar Tipo da Coluna.....	6
Mudar o Nome da Coluna.....	6
Excluir uma Coluna.....	6
Atualizar Dados na Tabela.....	6
ATENÇÃO!.....	7
Excluir Dados Fisicamente na Tabela.....	7
ATENÇÃO!.....	7

Introdução e Avisos

SQL é a Linguagem de Consulta Estruturada, é usada para armazenar, consultar, atualizar e gerenciar dados em Bancos de Dados Relacionais.

SGBD é um Sistema de Gerenciamento de Banco de Dados, exemplos: Oracle Database, PostgreSQL, MySQL, MariaDB, SQLite e outros

Palavras-chave e comandos não são *Case Sensitive*, ou seja, não diferenciam maiúsculas de minúsculas, mas por padrão os comandos são escritos com letras maiúsculas para melhor legibilidade.

Nomes de tabelas e colunas, a depender do SGBD, serão consideradas a diferença entre maiúsculas e minúsculas.

Observe que essa apostila estará mais voltada para o SGBD PostgreSQL do que para outros como Oracle SQL. É importante lembrar que mesmo a maioria dos SGBDs tendo como base o SQL ANSI, eles podem ter diferenças.

O ponto e vírgula (;) é um delimitador de comando.

Atenção! É necessário um SGBD para trabalhar com SQL.

Trabalhando com SQL

Criar Banco de Dados

Abra seu SGBD e uma janela de consulta

```
CREATE DATABASE nome_do_banco;
```

Exemplo real:

```
CREATE DATABASE rpd;
```

Deletar Banco de Dados

```
DROP DATABASE nome_do_banco;
```

Exemplo real:

```
DROP DATABASE rpd;
```

Criar Tabela

```
CREATE TABLE nome_da_tabela(  
    atributo_identificador INT PRIMARY KEY,  
    atributo1 VARCHAR(quantidade) NOT NULL,  
    atributo2 VARCHAR(quantidade),  
    atributo3 INT  
);
```

PRIMARY KEY (chave primária) é uma *constraint* (restrição) que indica que esse campo/atributo é um identificador, é ele quem identifica uma determinada coisa para a consulta, um identificador nunca deve se repetir na tabela.

INT indica tipo de dado inteiro

VARCHAR indica que esse campo recebe caracteres, e necessita que uma quantidade seja definida

quantidade delimita a quantidade de caracteres daquele determinado campo

NOT NULL é a *constraint* que indica que aquele campo nunca deve estar nulo/vazio, ou seja, quando os dados forem preencher essa tabela, esse campo obrigatoriamente deve ser preenchido

Exemplo real:

```
CREATE TABLE usuario(  
    id INT PRIMARY KEY,  
    nome VARCHAR(quantidade) NOT NULL,  
    email VARCHAR(quantidade),  
    idade INT  
);
```

Para o identificador, no PostgreSQL, usa-se **SERIAL** que é um valor do tipo inteiro mas que se auto incrementa, exemplo:

```
id SERIAL PRIMARY KEY
```

ATENÇÃO!

Muitos casos de tabela de usuário tem um campo CPF, embora esse campo leve um valor que identifica uma pessoa, ou seja, não se repete assim como o identificador, não é recomendado usá-lo para identificar um usuário, pois é uma informação pessoal que não deve ser exposta, e para fazer requisições seria necessário usá-lo, podendo vazar esse dado em uma URL ou corpo de requisição.

Deletar Tabela

```
DROP TABLE nome_da_tabela;
```

Exemplo real:

```
DROP TABLE usuario;
```

Inserir Dados na Tabela

```
INSERT INTO nome_da_tabela (atributo1, atributo2, atributo3) VALUES ('texto', 'texto', 10);
```

VALUES indica quais valores serão entregues, isso deve ser passado conforme a ordem de atributos declarados entre parênteses, ou irá preencher informações em posições erradas e resultar em problemas.

Exemplo real:

```
INSERT INTO usuario (nome, email, idade) VALUES ('Rebecca Chambers', 'rc@gmail.com', 18);
```

É importante notar que neste exemplo o atributo email poderia não ser informado, já que não viola uma regra de Não Nulo, diferente do nome.

Atualizar Estrutura da Tabela

Adicionar Coluna

```
ALTER TABLE nome_da_tabela ADD novo_atributo INT;
```

Exemplo real:

```
ALTER TABLE empresa ADD cnpj INT;
```

Modificar Tipo da Coluna

```
ALTER TABLE nome_da_tabela MODIFY COLUMN  
novo_atributo VARCHAR(14);
```

Exemplo real:

```
ALTER TABLE empresa MODIFY COLUMN cnpj VARCHAR(14);
```

Mudar o Nome da Coluna

```
ALTER TABLE nome_da_tabela RENAME COLUMN nome_antigo  
TO nome_novo;
```

Exemplo real:

```
ALTER TABLE empresa RENAME COLUMN cpf TO cnpj;
```

Excluir uma Coluna

```
ALTER TABLE nome_da_tabela DROP COLUMN  
atributo_errado;
```

Exemplo real:

```
ALTER TABLE empresa DROP COLUMN cpf;
```

Atualizar Dados na Tabela

```
UPDATE nome_da_tabela SET nome_atributo = 'novo  
valor' WHERE condicao;
```

WHERE (onde) essa cláusula indica qual registro deve ser atualizado através de uma condição.

ATENÇÃO!

Um UPDATE nunca deve ser realizado sem WHERE, caso contrário, todos os registros (linhas na tabela) dessa tabela terão seus dados alterados, e não apenas o registro alvo

Exemplo real:

```
UPDATE usuario SET email = 'rebeccachambers@rpd.com'  
WHERE id = 1;
```

Imaginando que a usuária Rebecca tenha o id como 1, apenas ela terá o seu email modificado. Se esse comando fosse executado sem WHERE, todos os outros usuários também teriam seu email atualizado para **rebeccachambers@rpd.com**

Dessa forma fica mais fácil compreender para que serve o id.

Excluir Dados Fisicamente na Tabela

```
DELETE FROM nome_da_tabela WHERE condicao;
```

Exemplo real:

Imaginando que em nossa tabela de usuário temos os usuários Rebecca Chambers com id 1, Chris Redfield com id 2 e Albert Wesker com id 3, e queremos excluir um desses dados, temos

```
DELETE FROM usuario WHERE id = 3;
```

Obtemos como resultado a exclusão real de Albert Wesker, ou seja, ele deixa de existir nessa tabela, mas não significa que o próximo usuário cadastrado poderá ter id 3, isso jamais acontecerá

ATENÇÃO!

Caso haja algum dado que está relacionado com este usuário, o sistema não permitirá que ele seja apagado

Nunca execute esse comando sem a cláusula WHERE, caso contrário, todos os dados da tabela selecionada serão deletados.